

Time : 3 hours

समय : 3 घंटे

Maximum Marks : 150

अधिकतम अंक : 150

QUESTION PAPER SPECIFIC INSTRUCTIONS

प्रश्न पत्र के लिए विशिष्ट अनुदेश

(Please read the following instructions carefully before attempting questions)

(कृपया प्रश्नों का उत्तर देने से पहले निम्नलिखित निर्देशों को ध्यान से पढ़ें)

1. There are fifteen (15) questions in all.
कुल पंद्रह (15) प्रश्न हैं।
2. All questions are compulsory.
सभी प्रश्न अनिवार्य हैं।
3. No extra/additional sheet will be provided.
कोई अतिरिक्त / विशेष शीट प्रदान नहीं की जाएगी।
4. Answer must be written in the authorized medium. No marks will be given for answers written in a medium other than the authorized one.
उत्तर अधिकृत माध्यम में ही लिखना होगा। अधिकृत माध्यम के अलावा किसी अन्य माध्यम में लिखे गए उत्तरों के लिए कोई अंक नहीं दिए जाएंगे।
5. All questions carry equal marks i.e. 10 each.
सभी प्रश्न समान अंक वाले हैं, अर्थात् प्रत्येक 10 अंक का है।
6. Each Question should be answered within the space provided for that particular question. Otherwise, the answer given will not be evaluated.
प्रत्येक प्रश्न का उत्तर उसके लिए निर्धारित स्थान पर ही लिखा जाए। अन्यथा, दिए गए उत्तर का मूल्यांकन नहीं किया जाएगा।

1. Consider a programming language that allows nested comments in the following format:
 comments begin with “(” and end with “)”, and they may be arbitrarily nested (i.e., a comment may contain another properly formed comment). For example, “(comment (nested) still in comment)” is valid. Write a detailed justification of whether such a language feature can be handled by a regular expression-based lexical analyzer (i.e., finite automata), or if a more powerful mechanism is required. Explain your reasoning based on formal language theory.
 10 Marks
2. Let L be a language over the alphabet $\{a, b\}$ such that $L = \{ w \in \{a, b\}^* \mid w \text{ has an equal number of substrings "ab" and "ba" } \}$. Is L regular, context-free, or neither? Justify your answer.
 10 Marks
3. Distinguish between cycles and circuits in both directed and undirected graphs. Explain how the detection of these structures varies depending on graph type and representation. Also explain how cycle detection plays a critical role in applications such as deadlock detection, scheduling, and electronic circuit design.
 10 Marks
4. Explain the end-to-end principle in network design. Evaluate its impact on the evolution of the Internet architecture and its interaction with middleboxes such as NATs and firewalls. Discuss how the proliferation of these intermediary devices challenges the principle, and critically assess whether the modern Internet still adheres to the end-to-end philosophy. Reflect on the future implications of this shift for protocol design and innovation.
 10 Marks
5. Python supports multiple programming paradigms – procedural, object-oriented, and functional. Critically evaluate how Python achieves this flexibility, and discuss the design consequences of supporting such diverse paradigms. How do these features influence maintainability, performance, and software architecture, particularly in large-scale systems or collaborative development environments?
 10 Marks

6. Python's module and package system enables scalable and modular software development. Analyze how the import mechanism, namespace resolution, and module caching contribute to both power and pitfalls in real-world Python applications. In what ways can improper use of modules hinder maintainability and introduce hard-to-detect bugs? Suggest best practices to mitigate such issues.

10 Marks

7. Flip-flops are fundamental building blocks of sequential digital circuits, used for memory storage and synchronization. Compare and contrast the various types of flip-flops – SR, D, JK, and T – in terms of their design, behavior, and practical applications. How does the choice of flip-flop affect circuit timing, clocking strategy, and susceptibility to race conditions in synchronous systems?

10 Marks

8. A hypothetical processor uses a 3-stage instruction pipeline: Fetch (F), Decode (D), and Execute (E). Assume the following conditions:

- The processor operates at a clock rate of 1 GHz.
 - Each instruction takes exactly one clock cycle per stage.
 - The pipeline is ideal (no stalls or hazards).
 - 100 instructions need to be executed.
- (a) Calculate the total execution time to complete all 100 instructions using the pipelined processor.
 - (b) Calculate the total time to complete the same 100 instructions in a non-pipelined processor, where each instruction takes 3 clock cycles (1 each for fetch, decode, and execute).
 - (c) Compute the speedup achieved by pipelining.

10 Marks

9. Write a Python program that reads a list of integers from the user and finds the second largest unique number. Assume the list has at least two unique values.

- (a) Provide the complete Python code to solve the problem.
- (b) If the user enters the numbers: 5, 3, 9, 1, 5, 3, 8, what will be the output?
- (c) Briefly explain how your code ensures the correct result even when there are duplicate values in the list.

10 Marks

- ✓ 10. Design an efficient algorithm for finding all maximal cliques in a large sparse graph. Analyse it with respect to complexity, heuristics, and potential parallelization strategies.
10 Marks
- ✓ 11. Calculate the minimum transmission time for a 100MB file over a 100Mbps link with 20ms latency. Add a 1% packet loss and calculate expected retransmissions using TCP.
10 Marks
- ✓ 12. Given an array of 10^6 integers, design and analyze a radix sort algorithm that sorts in base-16. Compute the time complexity and estimate actual time for a modern CPU (2 GHz).
10 Marks
- ✓ 13. Design a schema and write SQL queries to support a university's research publication database, with features such as co-author tracking, citation counts, and publication ranking. Include complex joins, set operations, and aggregate functions.
10 Marks
14. Design a virtual memory system for a 64-bit architecture supporting multiple isolated processes and shared memory regions. Provide address translation schemes and page replacement policies.
10 Marks
- ✓ 15. Write a program to simulate a self-balancing binary search tree (e.g., Red-Black Tree or AVL Tree). Evaluate its performance under different insertion orders and justify the need for balancing in practical systems.
10 Marks
-